

MIRAI — MASTER BLUEPRINT v2.0

Transforming Enterprise Through AI Hackathon | May 11–19, 2026

Accessible AI-Powered Robot Arm Simulator + Community Task Marketplace

Project Name: *Mirai (未来 — "Future")*

Track: *Track 3 — Robotics & Simulation + Track 2 — AI Agents with Google AI Studio*

AI Engine: *Gemini 2.0 Flash + Gemini 2.0 Pro via Google AI Studio (multimodal: text + voice + image)*

Physics Engine: *Rapier.js (WASM, client 60fps) + MuJoCo (server-side validation, Google/DeepMind standard)*

Deadline: *May 19, 2026 — Live Demo at AI & Big Data Expo North America*

Venue: *San Jose McEnery Convention Center, CA, USA*

Team: *Solo*

Prize Target: *Track 3 Winner (\$2,000–\$5,000) + Gemini Award (best Gemini use)*

Blueprint Version: *v2.0 — updated May 12, 2026 (scope tightened, MuJoCo upgrade, multimodal AI added)*

TABLE OF CONTENTS

1. [What is Mirai?](#)
2. [The Problem It Solves](#)
3. [Real-World Application](#)
4. [Complete Feature Set](#)
5. [Full Tech Stack — 32 Technologies](#)
6. [System Architecture](#)
7. [System Design Deep Dive](#)
8. [Engineering Decisions & Trade-offs](#)

9. [Tech Stack Pros & Cons](#)
 10. [8-Day Build Timeline](#)
 11. [UI/UX Page Visualizations](#)
 12. [Wow Factor Analysis](#)
 13. [Why Hire This Engineer](#)
 14. [Judge Pitch](#)
 15. [Submission Checklist](#)
 16. [Risk Register](#)
 17. [Business Model](#)
 18. [Pre-Hackathon Setup](#)
-

DAILY PROGRESS TRACKER

Today: Tuesday, May 12, 2026 (Day 2 of 8) | **Deadline:** Monday, May 19, 2026 — 8:00 AM PST
(Philippine Standard Time)

Legend:  Done  Not Started  In Progress (today)

DAY 1 — May 11 (Sunday) • Foundation + 3D Engine

Scaffold & Environment

-  GitHub repo created: github.com/Mizunandayo/mirai
-  package.json with all frontend deps (R3F, drei, Rapier, React Flow, Framer Motion, Jotai, Gemini SDK)
-  vite.config.js, tsconfig.json, index.html created
-  tailwind.config.js, .gitignore, README.md, .env.example created
-  backend/.env with live Gemini API key
-  server/requirements.txt — MuJoCo 3.x (upgraded from PyBullet)
-  npm install — NOT yet run (deps defined but not installed)

- ❌ pip install -r server/requirements.txt — NOT yet run
- ❌ git init + first push to github.com/Mizunandayo/mirai

3D Engine

- ✅ src/components/ArmViewer.tsx — R3F canvas, orbit controls, lighting, Physics wrapper, Grid
- ✅ src/components/RobotArm.tsx — base cylinder, 2 arm segments, end-effector sphere (5-segment scaffold)
- ✅ src/components/App.tsx — root component created
- ✅ src/store/atoms.ts — Jotai atoms: armSegments, gripper, task, simulation, community, Gemini state

Backend

- ✅ server/main.py — FastAPI skeleton, CORS, health check endpoint
- ❌ isAdvancedModeAtom not yet added to atoms.ts (Simple/Advanced UI toggle)
- ❌ Actual segment click-to-select highlight (component exists, feature not coded yet)

Day 1 Critical Carry-Over: Run npm install first thing Day 2 before writing any more code.

DAY 2 — May 12 (Tuesday) · Arm Design Studio TODAY

Carry-Over from Day 1 (do these FIRST)

- ❌ npm install --legacy-peer-deps (unblocks all frontend dev)
- ❌ pip install -r server/requirements.txt (unblocks backend dev)
- ❌ git init ; git remote add origin https://github.com/Mizunandayo/mirai.git ; git add . ; git commit -m "init: Mirai scaffold v2.0" ; git push -u origin main
- ❌ Add isAdvancedModeAtom to atoms.ts (localStorage-persisted Simple/Advanced toggle)
- ❌ Confirm app renders in browser: npm run dev → localhost

Arm Designer

- ❌ Arm segment panel — add/remove/resize segments in sidebar
- ❌ Real-time joint limit arc visualization on 3D arm
- ❌ Reach envelope heatmap (sphere of reachable space rendered in R3F)
- ❌ Gripper library — parallel jaw, suction cup, magnetic (3D preview per type)
- ❌ Arm config save/load (JSON → local file via Tauri or download)
- ❌ Design validation — torque, payload, reachability feasibility check

- ❌ **Real-time BOM cost counter** — live price total updates as segments added/resized (\$0 → \$287)
- ❌ Jotai atoms wired to all arm state (replaces current placeholder atoms)

Day 2 Deliverable: Full arm designer live in browser. BOM counter shows \$0 → \$287 as you build.

DAY 3 — May 13 (Wednesday) · Task Editor (React Flow)

- ❌ React Flow canvas with custom node types
- ❌ MOVE node — x/y/z/speed inputs + 3D ghost preview in R3F viewport
- ❌ GRIP node — open/close/force inputs
- ❌ WAIT node — ms delay input
- ❌ LOOP node — count + nested blocks
- ❌ IF node — condition string + then/else branches
- ❌ Live 3D ghost arm preview as blocks are placed (arm moves in viewport while editing)
- ❌ Export task as portable JSON file
- ❌ Error highlighting — red nodes for impossible coordinates, yellow for near-limit moves
- ❌ Keyboard shortcuts: Ctrl+S (save), Ctrl+Z (undo), Space (simulate), Delete (remove node)

Day 3 Deliverable: Fully functional visual task programmer that exports valid task JSON.

DAY 4 — May 14 (Thursday) · Physics Simulation (Rapier WASM)

- ❌ Rapier rigid body setup for each arm segment (Box + Cylinder colliders)
- ❌ Joint constraints — revolute (rotating), prismatic (sliding) in Rapier

- ❌ Task executor — reads task JSON blocks, drives Rapier simulation frame-by-frame
- ❌ Playback controls — play / pause / rewind / step-frame / speed: 0.25x – 4x
- ❌ Timeline scrubber — click any frame to jump to that position
- ❌ Joint angle HUD overlay — J1–J5 angles + gripper state shown live during playback
- ❌ Physics metrics panel — torque, velocity, acceleration per joint
- ❌ Collision highlight — red mesh flash + auto-rewind to collision frame
- ❌ Path trail visualization — glowing trajectory line behind arm's end-effector
- ❌ Environment objects — add table, shelf, box, sock pile, drawer to workspace

Day 4 Deliverable: *Smooth 60fps physics simulation of any task JSON. Collision detection working.*

DAY 5 — May 15 (Friday) • Gemini AI Integration

Core AI

- ❌ FastAPI POST /ai/plan endpoint — Gemini Flash integration
- ❌ Arm-aware prompt construction (injects arm specs + workspace into every call)
- ❌ Structured JSON output parsing and validation from Gemini response
- ❌ Auto-populate React Flow canvas from Gemini-returned task JSON
- ❌ Conversational refinement — "make it slower at step 3" updates specific blocks
- ❌ Gemini Pro error diagnosis — explain why simulation failed in plain English
- ❌ Loading states + streaming indicator + error handling

Edge Features (Day 5 Priority)

- ❌ **Voice input** — record audio → Gemini multimodal → task program in one step (demo wow moment)
- ❌ **AI confidence score badge** — "87% confident — J3 near limit at step 4" shown on every result
- ❌ **Side-by-side mode** — simulation left, live generated code right, synced playback
- ❌ **Pre-flight safety check** — Gemini audits task before simulation, structured warning list, [Fix All with AI] button
- ❌ **Agentic ReAct loop** — Think→Act→Observe loop for complex multi-step tasks (Gemini Award signal)

- ❌ **Natural language arm designer** — "arm reaching 1.2m that lifts 500g" → Gemini generates full arm config

***Day 5 Deliverable:** Speak a task → agentic planning loop → pre-flight check → confidence score → simulation. Gemini also designs arms from natural language.*

DAY 6 — May 16 (Saturday) • Backend + MuJoCo + Export

Backend & Physics

- ❌ FastAPI backend deployed to Railway (Docker container)
- ❌ WebSocket endpoint WS /ws/simulate for MuJoCo frame streaming
- ❌ MuJoCo MJCF/URDF builder from arm config
- ❌ Task executor in MuJoCo (runs same JSON task blocks as client-side Rapier)
- ❌ Accuracy comparison: MuJoCo vs. Rapier → UI accuracy badge ("94% accurate")
- ❌ **Physics side-by-side replay** — Rapier (left) + MuJoCo (right) play simultaneously, divergence frames in red
- ❌ **Servo lifespan predictor** — MuJoCo torque data → predicted hours per joint at current duty cycle

Export

- ❌ Jinja2 code generation — Arduino .ino + Python .py templates
- ❌ BOM generator from arm config with live AliExpress/Amazon pricing
- ❌ URDF export (ROS2-compatible robot description)
- ❌ **QR code generator** — scan with phone → hosted BOM + code page instantly
- ❌ **Signed export** — SHA-256 hash header in every downloaded file (30 min to build)
- ❌ ZIP bundle download — one .zip: code + BOM + wiring diagram

***Day 6 Deliverable:** Dual physics side-by-side live. Servo lifespan shown. Signed code downloads. QR judges can scan on stage.*

DAY 7 — May 17 (Sunday) • Community + Famous Preloads + Bonus

Community (Browse-Only — Roadmap is full marketplace)

- ❌ Community browse page — grid view with 3D mini-previews (read-only, no auth needed)
- ❌ One-click import — Gemini adapts community task to YOUR arm specs + shows compatibility %
- ❌ Seeded library — 12 community tasks (sock folding, mail sorting, box packing, etc.)

Famous Preloads (Day 7 Priority)

- ❌ **Boston Dynamics inspection task** — pre-loaded, one-click import
- ❌ **Tesla Optimus box-stacking task** — pre-loaded, one-click import
- ❌ **Toyota laundry folding task** — pre-loaded, one-click import

Quality

- ❌ Full end-to-end test: design arm → speak task → pre-flight check → simulate → side-by-side → export → QR code scan
- ❌ Performance pass — confirm 60fps on mid-range hardware

Bonus (only if Day 5–6 ahead of schedule)

- ❌ **BONUS: WebSerial live control** (5h) — browser → Arduino USB → real servo moves on stage
- ❌ **BONUS: GIF simulation export** (2h) — one-click animated GIF for sharing on Twitter/Discord

Day 7 Deliverable: *Browsable community with famous tasks. Full E2E flow confirmed. Bonus: real arm moving on stage.*

DAY 8 — May 18–19 (Monday–Tuesday) • Polish + Demo Prep

Final Testing

- ❌ Full E2E test: design arm → speak task → pre-flight check → simulate → side-by-side replay → QR code scan
- ❌ Verify signed export SHA-256 hash renders correctly in downloaded .ino file

- ❌ Performance optimization — 60fps confirmed on mid-range laptop (the venue machine)
- ❌ Test voice input on demo hardware — mic permissions + test fallback to typed input

Deployment

- ❌ Deploy frontend to Vercel (production URL)
- ❌ Deploy FastAPI backend to Railway (production URL)
- ❌ Sentry error tracking live on both frontend and backend
- ❌ Demo mode — pre-loaded "sock folding" example as landing state

Submission

- ❌ Record demo video (2 min: design → voice → pre-flight → simulate → side-by-side → QR scan)
- ❌ Slide deck (5 slides: problem → solution → demo → market → impact)
- ❌ README.md final pass — clean, screenshots, live demo link
- ❌ GitHub repository cleanup — no .env, no node_modules, no debug logs committed
- ❌ **Submit to lablab.ai before 8:00 AM PST May 19 ← HARD DEADLINE**

Day 8 Deliverable: Live app at production URL. Demo video uploaded. Slide deck ready.
Submitted before 8:00 AM PST May 19.

OVERALL PROGRESS SNAPSHOT

Day	Date	Focus	Status
Day 1	May 11 (Sun)	Foundation + 3D Engine	🟡 Partial — scaffold done, npm install pending
Day 2	May 12 (Tue)	Arm Design Studio	🟢 In Progress — TODAY
Day 3	May 13 (Wed)	Task Editor (React Flow)	❌ Not Started
Day 4	May 14 (Thu)	Physics Simulation (Rapier)	❌ Not Started
Day 5	May 15 (Fri)	Gemini AI Integration	❌ Not Started
Day 6	May 16 (Sat)	Backend + MuJoCo + Export	❌ Not Started
Day 7	May 17 (Sun)	Community + Preloads + Bonus	❌ Not Started
Day 8	May 18–19 (Mon–Tue)	Polish + Demo Prep + Submit	❌ Not Started

🕒 *Time remaining from May 12 morning: ~7 days*

🔥 *First action right now: npm install --legacy-peer-deps — nothing else can be tested until this runs.*

WHAT IS MIRAI?

Mirai is a **browser-based AI-powered robot arm simulator** that makes robotics accessible to everyone — not just researchers with \$100,000 budgets.

A student in Manila, a maker in Lagos, a nurse in Cebu with repetitive folding tasks — they open a browser, design a robot arm in 3D, describe what they want it to do in plain English ("pick up socks and fold them into the drawer"), and Gemini AI generates the motion program. A real-time physics engine simulates the execution. They can download the code and build the real arm for under \$300.

Mirai is not just a simulator. It is a **community-driven robotics platform** — the GitHub of robot task programs. Users share programs, remix each other's work, rate designs, and collectively build the world's largest open library of robot arm tasks.

The name **Mirai (未来)** means "Future" in Japanese — because this is what the future of accessible robotics looks like: open, intelligent, and community-powered.

THE PROBLEM IT SOLVES

Problem Statement

Robotics is the most powerful physical automation technology in the world — and it is locked behind \$100,000 price tags, PhD requirements, and closed enterprise ecosystems.

The Real Scenarios

Scenario 1 — The Home Maker

You do laundry three times a week. Sorting, folding, putting away. That is 4 hours a week, 208 hours a year, stolen from your life by repetitive motion. A robot arm could do it. But the cheapest capable arm costs \$15,000. And programming it requires ROS knowledge, C++ fluency, and weeks of calibration. So you keep folding socks.

Scenario 2 — The STEM Student

You are a computer science student in Southeast Asia. You want to learn robotics. Your university cannot afford a lab. Simulation software costs \$5,000/seat (MATLAB, Gazebo Pro, NVIDIA Isaac). The free alternatives (ROS + Gazebo) take 40 hours of Linux setup before you write one line of robot code. You give up before you start.

Scenario 3 — The Small Business Owner

You run a small packaging business. You pack 500 boxes per day by hand. Your back hurts. You know automation exists but you cannot hire a robotics engineer (\$150K/yr) to set it up. You cannot buy an industrial robot (\$80K+). You are stuck.

Scenario 4 — The DIY Maker

You want to build a robot arm. You find tutorials. They contradict each other. The parts lists are incomplete. The code does not work. You spend \$400 and 3 weeks and end up with a broken mess. You never try again.

What Exists Today

Solution	Price	Skill Required	Accessible?
Universal Robots (UR5e)	\$35,000	Robotics engineer	✗ No
NVIDIA Isaac Sim	\$5,000/seat	PhD-level	✗ No
ROS + Gazebo	Free	40+ hours setup	✗ No
Webots	Free	Programming required	✗ No
CoppeliaSim (V-REP)	\$3,000	C++ required	✗ No
Boston Dynamics Spot	\$75,000	Enterprise only	✗ No
Arduino robot kits	\$50-\$300	Some coding	⚠ Partial
Mirai	\$0 (simulator)	None	✓ Yes

What Mirai Does Differently

- **Zero setup** — runs in a browser, no install, no configuration
- **Natural language** — describe the task, Gemini generates the program
- **Real physics** — Rapier WASM engine validates your design before you spend \$1
- **Community-powered** — 1,000s of shared programs, all remixable
- **Bridge to reality** — export code → download DIY guide → build for under \$300
- **Designed for individuals** — not enterprises, not PhD programs, not research labs

REAL-WORLD APPLICATION

Who Uses Mirai

Primary Users (B2C — Individuals)

| User Type | Problem | How Mirai Solves It |

---|---|---

| Home makers | Repetitive physical tasks (laundry, sorting) | Design + simulate arm for that exact task |

| STEM students | No access to robotics lab | Browser-based simulation, zero cost |

| DIY makers | Failed robot builds, incomplete guides | Simulate first, then build with complete guide |

| Freelance engineers | Need to prototype robot tasks for clients | Test in simulation before committing to hardware |

| Small business owners | Manual repetitive tasks, can't afford industrial robots | \$300 DIY arm programmed exactly for their workflow |

| Content creators | YouTube/TikTok robot build content | Simulation preview for videos, community sharing |

Secondary Users (B2B — Small Scale)

| User Type | Problem | How Mirai Solves It |

---|---|---

| STEM teachers | No budget for physical robot labs | Class simulations, student community sharing |

| Repair shops | Repetitive part sorting/placement | Custom arm simulation, downloadable build guide |

| Small manufacturers | Semi-automation on a budget | Simulate the arm before the \$300 build investment |

Real Impact Metrics

- **Time saved:** A laundry-sorting arm saves 208 hours/year per household
 - **Cost reduction:** \$300 DIY arm vs. \$15,000 commercial equivalent = **98% cost reduction**
 - **Skill democratization:** Zero coding required → enables 500M non-programmers to design robots
 - **Educational access:** 130M students globally in underfunded STEM programs get a real robotics lab
 - **Waste reduction:** Simulate before building → eliminates \$300-\$400 failed builds
-

COMPLETE FEATURE SET

CORE — ARM DESIGN STUDIO

Feature	Description	Priority
3D Arm Builder	Drag-and-drop segment editor, real-time 3D preview	Core
Joint Configuration	Revolute, prismatic, fixed joint types with visual range indicators	Core
Segment Customization	Length, mass, material, color per segment	Core
Gripper Library	Parallel jaw, suction cup, magnetic, custom — visual 3D preview	Core
Design Validation	Real-time feasibility check (reachability, weight, torque)	Core
Design Templates	5 starter templates: 3-DOF, 5-DOF, SCARA, Delta, Cartesian	Core
Constraint Visualization	Joint limits shown as arc overlays on 3D arm	Enhanced
Reach Envelope	3D heatmap showing where arm can and cannot reach	Enhanced
Center of Gravity	Real-time CG calculation shown as visual indicator	Enhanced
Export to URDF	Standard Robot Description Format — ROS2 compatible	Enhanced
Real-Time BOM Cost Counter	As segments are added/resized, build cost updates live in the corner: \$0→\$47→\$143→\$287. Makes affordability story visceral, not a slide stat.	Core
Natural Language Arm Designer	Tell Gemini: "I need an arm reaching 1.2m that lifts 500g" → Gemini designs the full arm config (segments, joints, torques, gripper). AI designs hardware, not just software.	Core
Build-for-Budget Mode	Input a budget (\$50/\$150/\$300) → Gemini + BOM logic generates the best possible arm spec for that price. Serves students and home makers directly.	Enhanced

CORE — TASK EDITOR

Feature	Description	Priority
Visual Block Programming	Drag-and-drop nodes: MOVE, GRIP, WAIT, IF, LOOP, DETECT	Core
Node Properties Panel	Click any block to configure parameters (x/y/z, speed, force)	Core

Feature	Description	Priority
Live 3D Ghost Preview	As blocks are placed, 3D arm shows predicted position	Core
Task JSON Export	Save task as portable JSON, loadable on any arm configuration	Core
Task Versioning	Save v1, v2, v3 — compare side-by-side in simulation	Enhanced
Branching Logic	IF-ELSE visual branching — arm does different things based on sensor input	Enhanced
Error Highlighting	Red nodes for impossible moves, yellow for risky moves	Core
Hotkeys	Ctrl+S (save), Ctrl+Z (undo), Space (simulate), Delete (remove node)	Enhanced

CORE — SIMULATION ENGINE

Feature	Description	Priority
Real-Time Physics	Rapier WASM at 60fps in-browser, no server needed	Core
Collision Detection	Arm vs. objects vs. workspace boundaries	Core
Smooth Playback	Play, pause, rewind, step-frame, speed control (0.25x–4x)	Core
Timeline Scrubbing	Click any point on timeline to jump to that frame	Core
Joint Angle Display	Live HUD: J1–J5 angles + gripper state during playback	Core
Physics Metrics	Torque, velocity, acceleration per joint displayed	Enhanced
Accuracy Validation	Server-side MuJoCo validates client-side Rapier result (dual physics)	Core
Dual Physics Banner	Prominent UI badge: "Rapier ✅ 94% accurate MuJoCo validated" — makes the key innovation visible to judges	Core
Collision Replay	Auto-rewinds to collision point, highlights collision zone in red	Core
Path Visualization	Glowing trajectory trail showing arm's movement path	Enhanced
Environment Objects	Add table, shelf, box, sock pile, drawer to workspace	Core
Famous Task Preloads	3 seeded tasks: Boston Dynamics inspection, Tesla Optimus box-stacking, Toyota laundry fold — one-click import + Gemini adapts to your arm	Core

Feature	Description	Priority
Physics Side-by-Side Replay	After MuJoCo validation, Rapier (left) and MuJoCo (right) play simultaneously — divergence frames highlighted in red. Makes the dual physics architecture completely visible.	Core
Servo Lifespan Predictor	From MuJoCo torque data, predict each servo's operational lifespan at current duty cycle. "J2: ~180hrs at this load — reduce speed 20% to reach ~540hrs". No simulator has ever done this.	Enhanced

AI — GEMINI INTEGRATION

Feature	Description	Priority
Natural Language Task Generation	Type "fold socks into drawer" → Gemini generates block program	Core
Voice Input (Gemini Multimodal)	Speak your task aloud → Gemini transcribes + plans motion in one step	Core
Image Input (Gemini Multimodal)	Point camera at workspace → Gemini sees layout and plans task around real objects	Core
Arm-Aware Planning	Gemini reads YOUR arm specs, generates motions within YOUR constraints	Core
Multi-Step Reasoning	Complex tasks broken into logical phases automatically	Core
AI Confidence Score	Every generated task shows confidence: 87% — with one-sentence explanation of risks	Core
Side-by-Side Mode	Split view: left = simulation running live, right = code updating in real time as arm moves	Core
Pre-Flight Safety Check	Before any simulation runs, Gemini audits the task and shows a checklist: torque warnings, path boundary violations, gripper force overloads. [Fix All with AI] one-click repair.	Core
Agentic ReAct Loop	Multi-step Gemini agent: Think→Act→Observe loop. For complex tasks like "sort socks by color", agent adds DETECT block, generates IF/ELSE branches, verifies each step before moving to next. Real agent architecture, not single-prompt.	Core

Feature	Description	Priority
Gemini Arm Advisor	After MuJoCo finds torque overloads, Gemini recommends a hardware upgrade: "Upgrade J2 from MG995→MG996R (\$4 more) for 40% more torque headroom. New BOM: \$291." BOM updates live.	Enhanced
Optimization Suggestions	"Your task could be 30% faster if you change block 3 to speed 2x"	Enhanced
Error Diagnosis	Task fails in simulation → Gemini explains why + suggests fix	Enhanced
Conversational Refinement	"Make it slower around the sock pile" → AI updates specific blocks	Enhanced
Gemini Physics Tutor	After simulation, Gemini explains the physics in plain language: "J2 struggles because torque = force × distance. Shorten the forearm or upgrade the servo." Targets STEM students.	Enhanced
Natural Language Q&A	"Why is joint 3 at its limit?" → AI explains in plain English	Bonus
Task Translation	Import community task → Gemini adapts it to YOUR arm specs	Enhanced

COMMUNITY — MARKETPLACE

Feature	Description	Priority
Task Library	Browse 1,000+ community programs by category, rating, arm type	Core
3D Mini Preview	Hovering a task shows animated 3D preview thumbnail	Core
One-Click Import	Import any community task into your arm (auto-adapts to your specs)	Core
Creator Profiles	Each user has a profile: tasks created, stars received, builds confirmed	Enhanced
Star Rating	1–5 star community rating on each task	Core
Comments & Discussion	Thread-based comments on each task	Enhanced
Remix + Fork	"Fork" any task, create your own modified version	Enhanced
Build Confirmation	Users who built the real arm can mark task as "Real Build Verified"	Enhanced
Collections	Curated sets: "Top 10 Kitchen Tasks", "Beginner Tasks", "SCARA Only"	Enhanced
Search & Filter	Search by: arm type, DOF count, category, difficulty, verified builds	Core

EXPORT — DIY BRIDGE

Feature	Description	Priority
Bill of Materials Generator	Based on arm config → exact parts list + supplier links + prices	Core
Arduino Code Export	Ready-to-upload .ino file for Arduino Mega/Nano	Core
Python Script Export	Clean Python script compatible with common servo libraries	Core
MicroPython Export	For Raspberry Pi Pico / ESP32 builds	Enhanced
URDF Export	ROS2-compatible robot description file	Enhanced
3D Print Files	Auto-generated STL files for arm brackets, joints, gripper	Enhanced
Assembly Guide PDF	Step-by-step illustrated assembly instructions (auto-generated)	Enhanced
Wiring Diagram	Auto-generated servo wiring diagram from arm config	Enhanced
Cost Estimator	Total build cost with current Amazon/AliExpress pricing	Core
QR Code Export	Scan with phone → opens hosted BOM + code instantly. Makes build reality tactile for judges	Core
Bundle Download	One ZIP: code + BOM + wiring diagram	Core
Signed Export	Every downloaded file has a SHA-256 header: task hash + MuJoCo accuracy % + Gemini confidence %. Tamper-proof. Enterprise safety signal: "The code is exactly what was validated."	Core
GIF Simulation Export	One-click export of the Rapier simulation as an animated GIF. Share on Twitter/Discord. Viral loop: every shared GIF is free marketing. Demo: "47 people shared simulations today."	Enhanced
Assembly Guide PDF	Step-by-step illustrated assembly instructions (auto-generated)	Roadmap
3D Print Files / OpenUSD	STL + USD export for professional toolchain	Roadmap
WebSerial Live Control	Browser → real Arduino via USB Web Serial API. Simulation commands sent directly to physical servos. Turns simulator into live robotics controller. The #1 edge if demo hardware available.	Bonus (Day 7–8 if ahead)

#	Category	Technology	Version	Purpose	Why Chosen
1	Desktop Framework	Tauri v2	2.x	Native desktop app — transparent overlay, file system access	Rust security, 50MB vs. Electron 200MB
2	Frontend	React 19	19.x	Component-based UI with concurrent rendering	Concurrent mode for physics + UI
3	Language	TypeScript 5.x	5.x	Strict type safety across entire frontend	Catch physics math errors at compile time
4	Styling	TailwindCSS v4	4.x	Utility-first CSS, zero runtime	Fastest styling in 2026
5	Animation	Framer Motion v11	11.x	Spring physics UI animations	Matches physics theme of app
6	State	Jotai v2	2.x	Atomic state management for 3D scene + UI	Minimal re-renders during 60fps physics loop
7	3D Rendering	React Three Fiber	8.x	Declarative Three.js in React	React paradigm in 3D — unique DX advantage
8	3D Helpers	@react-three/drei	9.x	Gizmos, orbit controls, shaders, text	Saves 40+ hours of Three.js boilerplate
9	Physics (Client)	@react-three/rapier	0.12.x	Rapier WASM physics in React Three Fiber	Rust-compiled, fastest browser physics alive
10	Physics (Server)	MuJoCo	3.x	Server-side physics validation + accurate simulation	Google/DeepMind standard — free since 2022, more accurate than PyBullet, actively maintained
11	Visual Programming	React Flow v12	12.x	Node-based task editor (block programming)	Best visual graph library for React
12	Build Tool	Vite 6	6.x	Frontend bundling + HMR	Fastest dev cycle
13	AI Planner	Gemini 2.0 Flash	latest	Low-latency task generation (<500ms)	Hackathon requirement, genuinely best model

#	Category	Technology	Version	Purpose	Why Chosen
14	AI Reasoning	Gemini 2.0 Pro	latest	Complex multi-step robot task decomposition	Deeper reasoning for hard tasks
15	AI SDK	@google/generative-ai	latest	Official Google SDK for Gemini	Official, typed, supported
16	API Backend	FastAPI 0.115	0.115.x	Async Python API — simulation endpoints + WS	Fastest Python web framework
17	Python Runtime	Python 3.12	3.12.x	Backend + physics server	Latest stable, best performance
18	WebSocket	FastAPI WebSocket	built-in	Stream simulation frames to frontend	Real-time physics frame streaming
19	Robot Format	URDF	—	Universal Robot Description Format	Industry standard — ROS2, Isaac, Gazebo compatible
20	Code Gen	Jinja2	3.x	Template engine → Arduino/Python/MicroPython code output	Battle-tested, fast, logic-capable templates
21	Database	SQLite + SQLAlchemy	2.x	Community tasks, user profiles, ratings	Zero-config, works locally + cloud
22	Auth	JWT (python-jose)	3.x	Stateless auth for community features	No session management complexity
23	File Storage	Local FS + GitHub Releases	—	Task JSON files, STL files	Free, version-controlled, distributable
24	PDF Generation	WeasyPrint	62.x	Auto-generate DIY assembly guides as PDF	Python-native, CSS-styled PDFs
25	3D Export	OpenUSD (pxr)	24.x	Universal Scene Description for robot models	Apple/Pixar/NVIDIA standard — future-proof
26	Deployment	Vercel	latest	Frontend static deployment + serverless	Zero config, edge network
27	Deployment	Railway	latest	FastAPI + PyBullet backend	Simple Python container hosting

#	Category	Technology	Version	Purpose	Why Chosen
28	Containerization	Docker	27.x	Reproducible backend environment	Consistent between dev and prod
29	Version Control	Git + GitHub	—	Source control + release management	Public repo for submission
30	CI/CD	GitHub Actions	—	Auto-deploy on push	Professional practice
31	Monitoring	Sentry	latest	Error tracking frontend + backend	Catch runtime errors in judging session
32	Analytics	Plausible (self-hosted)	latest	Privacy-respecting usage analytics	GDPR compliant, no Google dependency

SCOPE DISCIPLINE — WHAT SHIPS VS. WHAT IS ROADMAP

A tight, polished 12-feature demo beats a broken 32-feature demo every time.

The tech stack above represents the full vision. The table below is the law for Days 1–8.

MUST SHIP (Demo Day Non-Negotiables)

#	Feature	Why It Must Ship
1	Interactive 3D arm designer (R3F)	The foundation — everything else depends on it
2	Real-time BOM cost counter	Affordability story is visceral: judges watch \$0 → \$287 live
3	Natural language arm designer	Gemini designs hardware, not just software — nobody else does this
4	Gemini text task generation	The core wow — "type a sentence, watch the arm move"
5	Gemini voice input	Demo moment — judges remember it
6	Pre-flight safety check	Gemini audits task before simulation — enterprise trust signal
7	Agentic ReAct task loop	Multi-step AI agent, not single prompt — wins the Gemini Award
8	Rapier WASM 60fps simulation	The physics — proves design works before building
9	MuJoCo dual physics validation	The differentiator — no one else does this
10	Physics side-by-side replay	Makes dual physics architecture visible — the architecture IS the demo
11	Dual physics accuracy UI badge	Score shown prominently — judges see it immediately

#	Feature	Why It Must Ship
12	AI confidence score display	Safety awareness — enterprise judges love this
13	Arduino + Python code export	Bridge to reality — proves it's more than a demo
14	Signed export (SHA-256 hash)	Tamper-proof code — 30 min to build, powerful to explain
15	QR code export	Tactile wow — judge scans it with their phone on stage
16	Bill of Materials with prices	"Build for \$287" — makes the \$300 claim concrete
17	3 famous task preloads	Instant content — Boston Dynamics, Tesla, Toyota
18	Side-by-side simulation + code view	Visual polish — simulation left, code right, live sync

BONUS (Add if Day 5–7 ahead of schedule — ordered by impact)

Priority	Feature	Est. Hours
1st	WebSerial live hardware control	5h — browser controls real Arduino via USB. Turns simulator into live robotics platform. Demo-defining.
2nd	Servo lifespan predictor	2h — MuJoCo torque data → predicted servo life. No simulator has ever done this.
3rd	Gemini arm advisor	4h — AI recommends hardware upgrades when torque limits exceeded. BOM updates live.
4th	Build-for-budget mode	4h — Input \$50/\$150/\$300 budget → Gemini generates optimal arm for that price.
5th	GIF simulation export	2h — One-click animated GIF. Viral sharing loop.
6th	Gemini image input	3h — Point webcam at workspace → Gemini plans task around real objects.
7th	Offline PWA	2h — Works without internet. Serves low-connectivity STEM students globally.

ROADMAP (Post-Hackathon — Mention in Pitch, Don't Demo)

Feature	Timeline	Notes
Community marketplace (full)	Week 2	Browse mockup only for demo
OpenUSD export	Month 1	Mention as "professional bridge" in pitch
WeasyPrint PDF guide	Month 1	ZIP download is sufficient for demo

Feature	Timeline	Notes
3D print STL files	Month 1	Out of scope for 8 days
Tauri desktop binary	Month 1	Web demo is fine for judging
Plausible analytics	Month 2	Not needed for hackathon
Gemini image input	Day 6–7 bonus	Add if Day 5 Gemini is ahead of schedule

PITCH POSITIONING (Judges Remember These Lines)

"We're GitHub for robot tasks, Figma for arm design, and Vercel for deployment — but for physical robotics."

"500 million people have repetitive physical tasks. We gave them a robot arm that costs \$287 to build."

"I spoke to Mirai. It heard me. It planned the motion. The simulation ran in 60fps. Then I downloaded the Arduino code."

SYSTEM ARCHITECTURE

Full System Diagram

Data Flow — One Task Generation

Dual Physics Architecture (Key Innovation)

SYSTEM DESIGN DEEP DIVE

1. State Architecture (Jotai Atoms)

2. Physics Frame Streaming Protocol

3. Gemini Prompt Architecture

4. URDF Auto-Generation

5. Code Generation Pipeline

ENGINEERING DECISIONS & TRADE-OFFS

Decision 1: Dual Physics (Rapier WASM + PyBullet)

The Problem: Fast UX requires client-side physics. Accurate simulation requires server-side full IK.

The Decision: Run BOTH simultaneously.

- Rapier WASM in browser → instant visual feedback at 60fps

- PyBullet on server → accurate validation, runs in background, returns accuracy score

Trade-off Analysis:

| Approach | Speed | Accuracy | Cost |

|---|---|---|---|

| Client-only (Rapier) |  60fps |  ~85% accurate | \$0/request |

| Server-only (PyBullet) |  1-3s latency |  99% accurate | \$0.001/request |

| **Dual (Mirai approach)** |  60fps (Rapier) |  99% accurate (PyBullet validates) | \$0.001/validate |

Engineering Justification: This is exactly how real robotics toolchains work (fast preview simulator + accurate physics engine). NVIDIA Isaac uses the same pattern. Engineers at Boston Dynamics, Tesla Optimus, and Figure AI will immediately recognize this as production-grade thinking.

Why It Wins Judges: Nobody else in this hackathon will think of dual physics. Everyone else will pick one or the other.

Decision 2: React Three Fiber (R3F) vs. Raw Three.js

The Problem: 3D rendering in React apps.

The Decision: React Three Fiber — declarative 3D.

Trade-off: R3F adds ~15KB bundle overhead but eliminates 40+ hours of imperative lifecycle management. For a hackathon, this is the correct trade.

Engineering Justification: React paradigm applied to 3D is genuinely novel. It makes the 3D scene reactive to state changes — when Jotai atoms update, R3F re-renders only the changed mesh. This is exactly how Vercel, Poimandres (R3F creators), and Three.js Journey teach modern 3D web dev.

Decision 3: Tauri v2 vs. Electron vs. Pure Web

The Problem: Desktop app or web app?

The Decision: Tauri v2 for desktop, Vercel for web — both from one codebase.

Metric	Electron	Tauri v2	Pure Web
App size	200MB+	15MB	N/A
File system access	✅ Yes	✅ Yes	❌ No
Native window controls	✅ Yes	✅ Yes	❌ No
Memory usage	500MB+	50MB	Browser limit
Security	❌ Node.js in renderer	✅ Rust backend IPC	Browser sandbox
Build complexity	Medium	Medium	Low

Engineering Justification: Tauri v2 allows Mirai to access the file system (save/load arm configs, export code files), use native window transparency for overlay mode, and keep the binary small enough to distribute freely. A 200MB Electron app cannot be given away free; a 15MB Tauri app can.

Decision 4: Jinja2 for Code Generation vs. LLM Code Generation

The Problem: How do we generate Arduino/Python code from task programs?

The Decision: Jinja2 templates for code generation, NOT Gemini.

Why Not LLM: LLM-generated code is non-deterministic. For robot arm firmware, a 1% hallucination rate means 1% of generated code could crash the arm into a wall. Templates guarantee correctness.

Why Jinja2: Deterministic, fast (<5ms), testable, auditable, versioned. The template IS the specification.

Engineering Justification: This is the same pattern AWS uses for CloudFormation templates, Kubernetes uses for Helm charts, and Terraform uses for infrastructure. Separation of structure (template) from data (task program) is a fundamental software engineering principle.

Decision 5: SQLite vs. PostgreSQL for Community

The Problem: Community database — what backend?

The Decision: SQLite for hackathon, PostgreSQL migration path documented.

Justification: SQLite handles 10,000 concurrent reads perfectly. The hackathon will not exceed this. Zero config SQLite means zero time spent on database setup. Post-hackathon migration to PostgreSQL is a one-line SQLAlchemy change.

Engineering Signal: Choosing the right tool for the scale you're at (not the scale you wish you had) is a senior engineering trait.

Decision 6: OpenUSD Export (Roadmap — Future-Proofing)

The Problem: Mirai needs to bridge from simulation to professional tools.

The Decision: Export robot scenes as OpenUSD in addition to URDF — **deferred to post-hackathon roadmap, not Day 1–8 scope.**

Why This Matters:

- OpenUSD is the universal 3D standard pushed by Apple, Pixar, NVIDIA Omniverse, Adobe
- NVIDIA Isaac Sim, Apple Vision Pro, and RealityKit all use OpenUSD natively
- A user who starts in Mirai can graduate to NVIDIA Omniverse with their exact arm config

Scope Decision: OpenUSD's pxr library is 500MB+ and has a steep API. For an 8-day hackathon, URDF export delivers 90% of the bridge-to-professional-toolchain story. OpenUSD is listed in README as roadmap.

Decision 7: MuJoCo over PyBullet for Server Physics

The Problem: Which Python physics engine should validate Rapier client-side results?

The Decision: MuJoCo — not PyBullet.

Why PyBullet Is The Wrong Choice In 2026:

- PyBullet's last major release was 2021. It is in maintenance mode.
- The robotics industry has converged on MuJoCo since Google/DeepMind made it free in October 2022
- A robotics judge from industry (Boston Dynamics, Tesla Optimus, Figure AI, 1X) will notice PyBullet immediately and view it as dated knowledge
- MuJoCo is more accurate, faster, and has an active development team

Why MuJoCo Is The Right Choice:

- Google/DeepMind standard — directly signals current industry knowledge
- Free and open source since 2022 (was \$5,000/seat before)
- Better contact simulation, more accurate inverse kinematics
- Used by: OpenAI (robotics), DeepMind (robotics), Boston Dynamics, Tesla Optimus, Figure AI
- pip install mujoco — same installation complexity as PyBullet

Trade-off: MuJoCo's Python API is slightly different from PyBullet's. Migration cost: ~4 hours on Day 6. The signal it sends to judges is worth 10x that cost.

Engineering Signal: Choosing MuJoCo over PyBullet is exactly the kind of "right tool at the right time" decision that separates engineers who read papers from engineers who only do tutorials.

Decision 8: Gemini Multimodal (Voice + Image Input)

The Problem: Text input lowers the bar. But can we go even lower?

The Decision: Gemini 2.0 natively supports audio and image input in the same API call. Use it.

Implementation:

Why This Wins The Gemini Award: The Gemini Award judges want to see Gemini's unique multimodal capabilities, not just text prompting. Every other team will do text. Voice + image input is a demo moment that takes 90 seconds to show and is completely unforgettable.

Demo Script Line: *"I held my phone up to my desk and showed Mirai where the sock drawer is. Gemini saw the real workspace and planned the task around it."*

Decision 9: WebSerial API — Browser Controls Real Hardware

The Problem: How do you prove a simulator is useful for real robots?

The Decision: Use the Web Serial API to send simulation commands directly to a real USB-connected Arduino from the browser.

Why It's The #1 Edge Feature:

- The app is no longer a simulator — it's a robotics control platform
- Zero install, zero drivers — Chrome on any OS handles it natively
- Demo moment: plug in USB on stage, simulation commands move real servos
- No other web-based robotics tool does this

Scope: Bonus feature Day 7–8. Requires one \$12 Arduino Nano + 3 hobby servos for demo. If hardware not available, WebSerial button is present but shows "Connect Hardware" — still signals the capability.

Engineering Signal: Web Serial is a 2024 Chrome API. Using it signals current platform knowledge. The feature requires zero backend, zero server — pure browser capability.

Decision 10: Agentic ReAct Loop for Complex Tasks

The Problem: Single-prompt Gemini → task JSON works for simple tasks. Complex tasks ("sort socks by color into separate drawers") require multi-step planning with feedback.

The Decision: Implement a ReAct (Reason + Act) agent loop where Gemini iteratively reasons, calls tools, observes results, and adjusts — rather than generating a complete program in one shot.

Why It Wins The Gemini Award: Every other team sends one prompt, gets one response, displays it. An agentic loop that reasons, uses tools, and self-corrects is exactly what "AI Agents" means. The Gemini Award evaluates agent architecture — this is the architecture they're looking for.

Decision 11: Pre-Flight Safety Check (Gemini as Safety Auditor)

The Problem: What happens when an AI-generated task would damage a real robot?

The Decision: Before every simulation run, a fast Gemini Flash call audits the task program against the arm spec and returns a structured warning list.

UI Output:

Why Enterprise Judges Love This: Any judge with production AI experience immediately asks "what happens when it's wrong?" This answers that question before they ask it. It signals safety-first engineering, not just demo engineering.

TECH STACK PROS & CONS

React Three Fiber + Rapier WASM

Pros	Cons
Declarative 3D — React paradigm in WebGL	R3F learning curve (~8 hours)
Rapier WASM = Rust-compiled, fastest browser physics	WASM bundle adds ~2MB to app size
Physics directly in React state machine	Complex arm IK requires manual solver
Zero server cost for client-side physics	WASM debugging tools less mature than JS
60fps guaranteed on modern hardware	

MuJoCo (Server-Side Validation) — UPGRADED FROM PyBullet

Pros	Cons
Google/DeepMind standard — the current industry reference	Slightly steeper API than PyBullet
Free and open source since 2022 (was \$5,000/seat before)	Python-only, no TypeScript types
More accurate contact simulation than PyBullet	Runs as separate process — Docker required
Actively maintained — commits from DeepMind robotics team weekly	Heavy dependency (~150MB container)
Used by: OpenAI, DeepMind, Boston Dynamics, Tesla Optimus team	
Judges with robotics backgrounds will immediately recognize it	

Gemini 2.0 Flash + Pro (Two-Model Strategy)

Pros	Cons
Flash: <500ms latency for task generation	Free tier: 15 req/min (sufficient for hackathon)
Pro: Deep reasoning for optimization	Pro: 1-3 second response time
Genuinely understands physics constraints	Hallucination risk on complex arm configs
Multimodal: can understand 3D images of arms	Rate limits during high traffic demo
Official hackathon sponsor — strong judging signal	

Tauri v2

Pros	Cons
15MB binary vs. Electron's 200MB	Rust backend requires Rust knowledge for deep customization
Native file system access (save arm configs, export code)	Smaller community than Electron
Transparent window overlay mode	Some Tauri v2 APIs still maturing
Rust IPC — no arbitrary Node.js code execution in backend	Cross-platform build matrix complex
Battle-tested since Miwa project — no learning curve	

React Flow v12

Pros	Cons
Production-ready node editor with 50K+ npm downloads/week	Limited 3D integration (2D node canvas)
Drag-drop nodes, custom node types, edges, connectors	Node layout can get messy for long programs
Undo/redo built-in	No built-in animation playback
Export as JSON natively	
Used by: Vercel, Linear, Buildkite for workflow editors	

OpenUSD Export

Pros	Cons
Industry standard — Apple, Pixar, NVIDIA all use it	pxr library is 500MB+ install
Bridges Mirai to NVIDIA Omniverse ecosystem	Complex API with steep learning curve
Future-proof — will be universal 3D format	Overkill for MVP — deferred to Day 7
Shows architectural foresight to senior engineers	

8-DAY BUILD TIMELINE

Day 1 (May 11) — Foundation + 3D Engine

Goal: Interactive 3D arm rendering in browser

Tasks:

- [] Create GitHub repo: github.com/Mizunandayo/mirai
- [] Scaffold Tauri v2 + React 19 + Vite 6 + TypeScript
- [] Install R3F + drei + Rapier + React Flow + Framer Motion + Jotai
- [] Render basic 5-segment arm in React Three Fiber
- [] Orbit controls (drag to rotate, scroll to zoom)
- [] Basic segment click-to-select highlight

Deliverable: Interactive 3D arm that rotates/zooms

Day 2 (May 12) — Arm Design Studio

Goal: Users can fully configure custom arms

Tasks:

- [] Arm segment panel (add/remove/configure segments)
- [] Real-time joint limit arc visualization on 3D arm
- [] Reach envelope heatmap (sphere of reachable space)
- [] Gripper library (parallel jaw, suction cup, magnetic)
- [] ARM config save/load (JSON to local file)
- [] Design validation (torque, payload, feasibility check)
- [] **Real-time BOM cost counter: live price updates as segments added/resized** ← affordability story live
- [] Jotai atoms for all arm state

Deliverable: Full arm designer with validation + live cost counter showing \$0 → \$287

Day 3 (May 13) — Task Editor (React Flow)

Goal: Visual block programming that outputs task JSON

Tasks:

- [] React Flow canvas with custom node types

- [] MOVE node: x/y/z/speed inputs + 3D position ghost preview
- [] GRIP node: open/close/force inputs
- [] WAIT node: ms input
- [] LOOP node: count + nested blocks
- [] IF node: condition + then/else branches
- [] Live 3D ghost arm preview as blocks are placed
- [] Export task as JSON
- [] Error highlighting (red nodes for invalid moves)

Deliverable: Fully functional visual task programmer

Day 4 (May 14) — Physics Simulation (Rapier WASM)

Goal: Real-time physics playback in browser at 60fps

Tasks:

- [] Rapier rigid body setup for each arm segment
- [] Joint constraints (revolute, prismatic) in Rapier
- [] Task executor: reads JSON blocks, drives Rapier simulation
- [] Playback controls: play/pause/rewind/speed/scrub
- [] Timeline component with frame markers
- [] Joint angle HUD overlay during playback
- [] Collision highlight (red mesh flash on contact)
- [] Path trail visualization (glowing trajectory line)

Deliverable: Smooth 60fps physics simulation of any task

Day 5 (May 15) — Gemini AI Integration

Goal: Full AI layer — voice, agentic loop, safety check, hardware design. This is the demo centerpiece.

Tasks:

- [] Gemini 2.0 Flash text integration (POST /ai/plan)
- [] Arm-aware prompt construction (injects arm specs + workspace into every call)
- [] Structured JSON output parsing from Gemini response
- [] Auto-populate React Flow canvas from Gemini output
- [] **Voice input: record audio → send to Gemini multimodal → task program ← demo wow**
- [] **AI confidence score badge: "87% confident — J3 near limit at step 4" ← safety signal**
- [] **Side-by-side mode: simulation left, live code right ← polish wow**

- [] **Pre-flight safety check: Gemini audits task before simulation, lists warnings, [Fix All with AI] button** ← enterprise trust
- [] **Agentic ReAct loop: Think→Act→Observe for complex multi-step tasks** ← Gemini Award signal
- [] **Natural language arm designer: "I need an arm that reaches 1.2m and lifts 500g" → Gemini generates full arm config** ← hardware AI
- [] Conversational refinement ("make it slower at step 3")
- [] Gemini Pro error diagnosis (explain why simulation failed)
- [] Loading states + error handling

Deliverable: Speak task → agentic planning → pre-flight check → confidence score. Gemini designs arms, not just tasks.

Day 6 (May 16) — Backend + MuJoCo Validation + Export

Goal: Accurate physics validation + code generation + QR + signed export + physics side-by-side

Tasks:

- [] FastAPI backend setup (Railway Docker)
- [] WebSocket endpoint for MuJoCo streaming (replaces PyBullet)
- [] MuJoCo MJCF/URDF builder from arm config
- [] Task executor in MuJoCo
- [] Accuracy comparison: MuJoCo vs. Rapier → UI accuracy badge
- [] **Physics side-by-side replay: Rapier (left) + MuJoCo (right) play simultaneously, divergence frames in red** ← architecture visible
- [] **Servo lifespan predictor: MuJoCo torque data → predicted hours per joint** ← engineer signal
- [] Jinja2 code generation: Arduino + Python
- [] BOM generator from arm config with live pricing
- [] **QR code generator: scan → hosted BOM + code page** ← tactile demo moment
- [] **Signed export: SHA-256 hash header in every downloaded file (30 min)** ← tamper-proof signal
- [] ZIP bundle download endpoint

Deliverable: Dual physics side-by-side live + servo lifespan shown + signed code + QR judges scan on stage

Day 7 (May 17) — Community Browse + Famous Preloads + Bonus Edge Features

Goal: Community presence + seeded content + ship bonus edge features if ahead of schedule

Scope discipline: Full community (ratings, comments, user profiles) is roadmap. Ship browse + import + 3 famous preloads only.

Tasks:

- [] **3 famous task preloads: Boston Dynamics inspection, Tesla Optimus box-stack, Toyota laundry** ← instant wow
- [] Community browse page: grid view with 3D mini-previews (read-only)
- [] One-click import (Gemini adapts community task to YOUR arm specs + shows compatibility %)
- [] Seeded library: 12 community tasks (sock folding, mail sorting, box packing, etc.)
- [] Full end-to-end test: design arm → speak task → pre-flight check → simulate → side-by-side → export → QR code
- [] Performance pass: 60fps on mid-range hardware confirmed
- [] **BONUS if ahead: WebSerial live control (5h) — browser → Arduino USB → real servo moves on stage**
- [] **BONUS if ahead: GIF export (2h) — one-click simulation GIF for sharing**

Deliverable: Full E2E flow working. If WebSerial done: real arm moves on stage during demo.

Day 8 (May 18/19) — Polish + Demo Prep

Goal: Production-ready for live judging on stage

Tasks:

- [] Full E2E test: design arm → speak task → pre-flight check → simulate → side-by-side replay → QR code scan
- [] Verify signed export hash shows correctly in downloaded .ino file
- [] Performance optimization (60fps on mid-range hardware)
- [] Sentry error tracking integration
- [] Deploy: Vercel (frontend) + Railway (backend)
- [] Demo mode: pre-loaded "sock folding" example for stage
- [] Test voice input on venue hardware (mic permissions, fallback to text if needed)
- [] Record demo video (2 min: design → voice → pre-flight → simulate → side-by-side → QR scan)
- [] Slide deck (5 slides: problem → solution → demo → market → impact)

- [] README + GitHub cleanup
- [] Submit to lablab.ai

Deliverable: Live production app + demo video + polished pitch ready for stage

UI/UX PAGE VISUALIZATIONS

PAGE 1: ARM DESIGN STUDIO

PAGE 2: AI TASK PLANNER (Gemini)

PAGE 3: PHYSICS SIMULATION PLAYBACK

PAGE 4: COMMUNITY LIBRARY

PAGE 5: EXPORT & DIY BRIDGE

WOW FACTOR ANALYSIS

What Makes a Senior Engineer Say "Who built this?"

1. DUAL PHYSICS ENGINE ARCHITECTURE (MuJoCo + Rapier)

"Wait — they're running Rapier WASM client-side at 60fps AND validating with MuJoCo server-side AND reconciling the two results into an accuracy score shown in the UI? MuJoCo — not PyBullet. They know the current standard. That's the exact pattern we use in our production robotics pipeline."

— Principal Engineer, Boston Dynamics

2. GEMINI MULTIMODAL — VOICE + IMAGE INPUT

"They didn't just do text prompting. They used Gemini's audio input to let users speak their task. And image input to let Gemini see the real workspace from a webcam. That's not 'I used the Gemini API.' That's genuine multimodal system design."

— AI Research Engineer, Google DeepMind

3. REACT THREE FIBER WITH JOTAI ATOMS DRIVING 3D STATE

"Declarative 3D scene bound to atomic state — when joint angles change in Jotai, only the affected mesh re-renders. Not the whole canvas. That's fine-grained reactivity applied to WebGL. I've never seen this in a hackathon project."

— Staff Engineer, Vercel

4. AI CONFIDENCE SCORE WITH SAFETY REASONING

"They showed a confidence score on every AI-generated task, with a one-sentence explanation of why it might fail on real hardware. That's not a UI gimmick — that's AI safety awareness applied to physical systems. Enterprise buyers care about this."

— Product Lead, Figure AI

5. JINJA2 CODE GEN (NOT LLM CODE GEN)

"They explicitly chose templates over LLM for code generation. Because they understand that robot firmware must be deterministic and auditable. A 1% LLM hallucination rate is acceptable for text; it's unacceptable for servo commands. That's a real engineering judgment call."

— Engineering Manager, Figure AI

6. QR CODE → JUDGE SCANS IT ON STAGE

"I've judged 40 hackathons. Nobody has ever handed me a QR code that opens the exact bill of materials for the robot I just watched simulated. That's \$287. I can order it right now. The bridge to reality just became physical."

— Hackathon Judge, AI & Big Data Expo

7. GEMINI TWO-MODEL STRATEGY

"Flash for speed, Pro for reasoning. Different models for different jobs in the same system. That's AI systems design, not just 'call the AI API'. And the prompts are arm-aware — Gemini knows the joint limits and reach envelope before planning motions. That's context injection done right."

— AI Research Engineer, Google DeepMind

8. FAMOUS TASK PRELOADS — BOSTON DYNAMICS, TESLA OPTIMUS, TOYOTA

"They seeded the library with the three most famous robot tasks in the industry. I can one-click import Tesla Optimus's box-stacking task, adapt it to my \$287 arm, and simulate it. That's not a demo. That's a product."

— Robotics Maker, DIY community

9. COMMUNITY IMPORT/ADAPT WITH GEMINI

"The import button doesn't just copy the task — it re-plans the motion for YOUR arm's constraints using Gemini. That's the difference between a file share and an intelligent platform. The network effects here are real."

— Product Engineer, GitHub

WHY HIRE THIS ENGINEER

What This Project Signals to Top Companies

To Google / DeepMind:

"This engineer used Gemini multimodal (text + voice + image) across two models (Flash + Pro) with arm-aware context injection, multi-turn conversation, confidence scoring, and structured JSON output. They understand LLM system design at the level we hire for."

To NVIDIA (Robotics / Omniverse):

"They output URDF. They understand our toolchain. The dual physics pattern (fast approximation + ground truth validation) is exactly how Isaac Sim works. They are already thinking like a robotics platform engineer."

To Boston Dynamics / Tesla Optimus / Figure AI / 1X:

"They chose MuJoCo over PyBullet. They know the current standard. They understand dual physics patterns. They built arm-aware AI planning with constraint validation. This person has genuine robotics engineering knowledge, not just tutorial knowledge."

To Tauri / Vercel / Modern Tooling Companies:

"React Three Fiber, Jotai, Tauri v2, Vite 6, TailwindCSS v4 — bleeding edge frontend. And they chose the RIGHT tool for each job: Rapier WASM for client physics, Jinja2 for codegen, SQLite for MVP, MuJoCo for server validation."

The Engineering Traits Demonstrated:

| Trait | Evidence |

|---|---|

| Systems thinking | Dual physics (fast preview + accurate validation) |

| Current knowledge | MuJoCo over PyBullet — knows the 2026 standard |

| Right-tool-for-the-job | Jinja2 not LLM for codegen; SQLite not Postgres for MVP |

| Multimodal AI design | Voice + image + text in one system, not just text |

| Safety awareness | AI confidence score with per-step risk explanation |

| Performance consciousness | Jotai atoms → only affected R3F mesh re-renders at 60fps |

| Security awareness | Tauri Rust IPC; JWT auth; no arbitrary JS execution in backend |

| Product thinking | Community marketplace with "Build Verified" badge; network effects |

| User empathy | Zero-setup, browser-based, natural language input — for everyone |

| Speed of execution | Full-stack complex system shipped in 8 days, solo |

COMPETITIVE INTELLIGENCE

What Other Track 3 Teams Will Submit

Based on every Track 3 / Robotics hackathon submission pattern in the last 3 years, here's what Mirai will be judged against:

Competitor Type	What They Build	Why Mirai Beats It
Unity/Unreal team	Hardcoded 3D animation with no real physics	Mirai uses real Rapier WASM + MuJoCo physics. Their "simulation" is a scripted video.
ROS + React wrapper	ROS2 running locally, React frontend that shows a terminal	Requires Linux + ROS install. Judges can't run it. Mirai runs in any browser tab.
"Chatbot + robot arm" team	GPT/Claude chatbot + pre-built arm library, no physics	No dual physics, no code export, no real constraint validation. Demo is a chat UI.
MuJoCo tutorial project	MuJoCo running server-side, no frontend, Jupyter notebook demo	No accessible UI. Judges have to install Python. No AI planning layer.
Figma-style drag interface	Visual arm designer, no simulation or AI	Pretty but can't move. No code output. No bridge to real hardware.

Mirai's Moat (What Cannot Be Copied In 8 Days)

- Dual physics with reconciliation** — Rapier WASM client-side + MuJoCo server-side + accuracy badge. This is a distributed systems architecture decision, not a feature.
- Arm-aware Gemini planning** — Gemini's prompt includes the exact joint limits, reach envelope, and payload spec of the user's custom arm. Every team will do "generate a robot task." Nobody else injects the arm's actual physics spec into the prompt.
- Deterministic code export pipeline** — Jinja2 templates generate Arduino/Python that will run on real hardware without modification. LLM-generated code always has a hallucination failure rate. Jinja2 doesn't.
- Gemini multimodal (voice + image)** — Speaking a task or showing a webcam frame of your real workspace. This is technically trivial with Gemini 2.0 but visually unforgettable. No competitor will think to add it.
- QR code → real BOM pricing** — The bridge from simulation to physical reality, deliverable in 90 seconds on stage. This demo beat is impossible to replicate without the full pipeline.

Judging Criteria Analysis

Criterion	Weight	Mirai's Advantage
Technical Complexity	High	Dual physics, WASM, multimodal AI, WebSocket streaming, code generation
Innovation	High	No browser-based arm design tool exists. Dual physics is novel for web. Voice → robot is novel.
Gemini Integration	High (Gemini Award)	Two models, multimodal, arm-aware context, confidence scoring
Practical Impact	Medium	\$287 build cost, Arduino export, QR BOM — literally buildable today
Presentation Quality	Medium	QR scan on stage, famous task preloads, 60fps live simulation
Track Fit (Robotics)	Required	Full Track 3 compliance: 3D, physics, URDF export, constraint validation

The One-Line Positioning

"Mirai is the only browser tool where you design a robot arm, speak your task, simulate it with two physics engines, and download the Arduino code to build it for \$287."

Every word of that sentence describes a capability no competitor will have.

JUDGE PITCH

The 3-Minute Pitch Script (May 19, Live Stage)

Opening (30 seconds — THE HOOK):

"500 million people have repetitive physical tasks a robot arm could handle. But the cheapest capable arm is \$15,000. Programming it requires a PhD. Simulation software costs \$5,000 a seat. So you keep folding socks. We built Mirai to fix that."

Problem + Market (20 seconds):

"130 million students in STEM programs with no robotics labs. 50 million DIY makers who want to build robots but fail because the tools are too hard. The robotics industry has ignored all of them. Until now."

Live Demo (90 seconds):

"Let me show you. This is Mirai — it runs in your browser. Right now."

- 1. Drag to build a 5-segment arm (20 seconds)*
- 2. **[VOICE]** Say aloud: "Pick up the sock and fold it into the drawer" (15 seconds)*
- 3. Gemini generates the motion program from voice (10 seconds)*
- 4. Hit Simulate — 60fps physics, MuJoCo accuracy badge appears (20 seconds)*
- 5. Show confidence score: "Gemini is 87% confident — joint 3 near limit at step 4" (10 seconds)*
- 6. **[QR CODE]** Hold up phone. Scan QR. BOM loads: \$287 total (15 seconds)*

Technology (20 seconds):

"Under the hood: Rapier WASM physics at 60fps client-side — validated by MuJoCo server-side, the same engine used by Google DeepMind and Tesla Optimus. Gemini 2.0 multimodal — voice, image, and text. Jinja2 code generation — deterministic, not hallucinated. URDF export for ROS2 and NVIDIA Omniverse."

Positioning + Closing (10 seconds):

"We're GitHub for robot tasks. Figma for arm design. Vercel for deployment. But for physical robotics."

Mirai. Future. In your browser. Today."

SUBMISSION CHECKLIST

Required by May 19

lablab.ai Submission:

- [] Project Title: Mirai — Accessible AI Robot Arm Simulator
- [] Short Description: Browser-based robot arm simulator powered by Gemini 2.0 multimodal. Design arms, speak your task, validate with MuJoCo + Rapier dual physics, and get a complete DIY guide to build it for under \$300.
- [] Long Description: Full problem statement + solution + tech stack + impact
- [] Technology Tags: Gemini, React Three Fiber, Rapier.js, FastAPI, Three.js, Tauri, MuJoCo
- [] Category Tags: Robotics & Simulation, AI Agents, Open Source, Education
- [] Cover Image: 1200x630px (arm simulation screenshot + Mirai logo)
- [] Demo Video: 2 min YouTube (design → voice → simulate → MuJoCo badge → QR scan → code)
- [] Slide Deck: 5 slides (problem / solution / demo / market / impact)
- [] GitHub Repo: github.com/Mizunandayo/mirai (public, clean, documented)
- [] Live Demo URL: mirai.vercel.app

Track Requirements:

- [] Track 3 (Robotics & Simulation): 3D arm designer + Rapier WASM + MuJoCo validation + URDF export
- [] Track 2 (Gemini): Gemini 2.0 Flash + Pro, multimodal (voice + image + text), arm-aware prompting
- [] Gemini Award: Two-model strategy + multimodal + confidence scoring + arm-aware context injection

RISK REGISTER

Risk	Probability	Impact	Mitigation
Rapier WASM arm IK complexity	● High	● Medium	Use simple forward kinematics first, add IK Day 5+
React Three Fiber learning curve	● High	● Medium	Day 1 dedicated to R3F basics, use drei helpers
Gemini rate limits during live demo	● Medium	● High	Cache last 10 Gemini results; pre-compute demo task

Risk	Probability	Impact	Mitigation
MuJoCo container setup delay	● Medium	● Medium	Day 6 allocated entirely to backend + Docker
Physics divergence (Rapier vs MuJoCo)	● Medium	● Low	Feature, not a bug — show accuracy score as key feature
3D performance on judge's laptop	● Medium	● Medium	Pre-record simulation as GIF fallback; LOD reduction
Time overrun on community features	● High	● Low	Community is Day 7 — ship browse-only if behind
Live demo crash on stage	● Medium	● High	Pre-recorded video backup + offline demo mode
Voice input mic permissions on demo device	● Medium	● Medium	Test on venue laptop Day 8; have typed text fallback

BUSINESS MODEL

Monetization Strategy

Market Size

Segment	TAM	SAM	SOM (Y1)
DIY makers + hobbyists	\$800M	\$120M	\$2M
STEM education	\$2.1B	\$300M	\$5M
Small business automation	\$5.4B	\$500M	\$8M
Indie robotics developers	\$400M	\$80M	\$1.5M
Total	\$8.7B	\$1B	\$16.5M

PRE-HACKATHON SETUP

Before May 11, 9:00 PM PST (Hackathon Start)

TIER 1 — CRITICAL (Do Now):

- [] GitHub repo created: github.com/Mizunandayo/mirai
- [] Gemini API key: <https://ai.google.dev> → "Get API key" → save in .env
- [] Vercel account created: vercel.com → sign up with GitHub
- [] Railway account created: railway.app → sign up with GitHub

TIER 2 — ENVIRONMENT:

- [] Install dependencies:
 - [] Tauri v2 project scaffold (copy from Miwa as base)
 - [] Test Gemini API key works:
 - [] Test Rapier physics works (basic rigid body falls under gravity)
 - [] Test R3F renders a rotating cube

TIER 3 — LEARNING (30 min each):

- [] React Three Fiber quickstart: <https://docs.pmnd.rs/react-three-fiber>
- [] @react-three/rapier demo: <https://github.com/pmndrs/react-three-rapier>
- [] React Flow intro: <https://reactflow.dev/guide/>
- [] Gemini structured output docs: https://ai.google.dev/docs/structured_output

IDENTITY

- **Project:** Mirai (未来 — "Future")
 - **Developer:** Francis Daniel (GitHub: Mizunandayo, Discord: Mizu)
 - **Hackathon:** Transforming Enterprise Through AI (lablab.ai)
 - **Tracks:** Track 3 — Robotics & Simulation / Track 2 — Gemini AI Agents
 - **Event:** AI & Big Data Expo North America, May 19, San Jose CA
 - **Repo:** <https://github.com/Mizunandayo/mirai>
 - **Live Demo:** <https://mirai.vercel.app>
-

KEY ENGINEERING PHILOSOPHY

"The best engineers don't just solve problems. They solve the right problems at the right level of abstraction, with the right tools, and they leave the architecture better than they found it."

Mirai is built on these principles:

1. **Simulate first** — never spend \$300 on a design you haven't validated
2. **Dual truth** — fast approximation for UX, ground truth for accuracy
3. **Deterministic where it matters** — Jinja2 codegen, not LLM, for firmware
4. **Open by default** — URDF + OpenUSD for maximum interoperability
5. **Community creates moat** — user programs make the platform more valuable than any single feature

Mirai Blueprint v1.0 — May 11, 2026

Built with the same engineering rigor as production systems, in 8 days, solo.